VisionInspect AI

Manufacturing Defect Detection & Quality Inspection System

Milestone 4 Report

Testing, Deployment & Documentation

Individual Contribution Report

Prepared by: Sarang Soni

Program: Infosys Springboard Internship 7.0

Domain: Artificial Intelligence / Machine Learning

Date: August 2026

Table of Contents

1. Work Summary

Milestone 3 established the trained PatchCore models and a working inference/serving layer for VisionInspect AI. For Milestone 4, my individual focus shifted to integrating that inference layer with the full-stack application, resolving the deployment issues that surfaced during integration, and packaging the entire platform for containerized, cross-platform deployment.

Working independently on the backend and deployment track, I took the application from a locally-running, manually-started FastAPI + React setup to a fully containerized system that starts with a single Docker Compose command and can be exposed publicly for remote evaluation via a Cloudflare Tunnel.

1.1 Modules Completed

- Backend startup automation — automatic demo-account seeding on application boot.
- Role-Based Access Control (RBAC) verification and correction for the /predict inference route.
- Dependency resolution for the AI/ML stack (PyTorch, Torchvision, Anomalib) inside the backend container.
- Cross-platform path handling fix in the inference module (predict.py) for Linux-based Docker execution.
- Heatmap image delivery fix — Base64 payload encoding and an OpenCV-based fallback overlay for PASS results.
- Full containerization of frontend and backend using Docker and Docker Compose.
- Public demo exposure using Cloudflare Tunnel for remote, browser-based evaluation.
- End-to-end functional testing of the deployed system: inspection console, owner dashboard, audit logs, and quality certificate generation.

1.2 Tools and Technologies Used

Category	Technology
Frontend	React (Vite), Tailwind CSS
Backend	FastAPI (Python), Uvicorn, SQLAlchemy
AI Model Engine	PyTorch, Torchvision, OpenCV, Anomalib (PatchCore)
Database	SQLite (via SQLAlchemy ORM)
Authentication	JWT + Role-Based Access Control
Containerization	Docker, Docker Compose
Public Access	Cloudflare Tunnel
Version Control	Git & GitHub

2. System Overview

VisionInspect AI is an AI-powered manufacturing defect detection and quality inspection platform. It combines an unsupervised anomaly detection model (PatchCore, via Anomalib) trained across the 15 MVTec AD product categories with a FastAPI backend handling authentication, role-based access, and inspection logging, and a React frontend providing the live inspection console, owner dashboard, and audit trail.

The five architectural layers are summarized below:

- Frontend: React (Vite, Tailwind CSS) — live inspection console, category selection, image upload, results/analytics dashboard, audit log.
- Backend: FastAPI (Python, Uvicorn, SQLAlchemy) — REST API routes, JWT authentication, RBAC, severity scoring, inspection logging.
- AI Model Engine: PyTorch, Torchvision, OpenCV, Anomalib (PatchCore) — anomaly detection and heatmap overlay generation.
- Database: SQLite via SQLAlchemy ORM — user accounts, roles, and full inspection history.
- Infrastructure: Docker and Docker Compose — consistent, cross-platform containerized deployment.

3. Individual Contribution: Testing & Deployment

My contribution in this milestone centered on taking the system built through Milestone 3 from a working prototype to a reliably deployable product. This involved backend hardening, dependency and environment fixes, and full containerization — the work summarized in the challenges below was carried out and verified independently.

4. Key Technical Challenges & Resolved Issues

Challenge 1: Database Seeding & Missing Demo Accounts

Issue: The application raised an error stating that the demo account 'supervisor' was missing, blocking login entirely.

Root Cause: Unsaved changes in main.py caused Docker to repeatedly rebuild from a cached image without properly initialized database tables.

Fix Implemented:

- Added a startup event, seed_demo_users(), in src/backend/main.py to automatically verify and seed the admin (admin/admin123) and supervisor (supervisor/supervisor123) demo accounts into SQLite on application startup.
- Reset the Docker volume state (docker compose down -v) and rebuilt the environment (docker compose up --build) to confirm the fix took effect cleanly.

Challenge 2: Role-Based Access Control (RBAC) Enforcement

Issue: Public registration defaulted new users to the factory_supervisor role, which prevented access to the AI inspection features.

Root Cause: The /predict route explicitly requires admin or quality_engineer privileges as a security control.

Fix Implemented:

- Used the auto-seeded admin credentials to obtain elevated privileges for verification.
- Confirmed the backend security scopes so that public users receive baseline roles while administrators and quality engineers retain authorization to run inference.

Challenge 3: Missing Container Dependencies (anomalib / torch)

Issue: Triggering an inspection from the UI raised a backend error: "No module named 'anomalib'".

Root Cause: The heavy machine-learning packages had been deliberately omitted from requirements.txt to keep initial container build times low.

Fix Implemented — added a CPU-optimized PyTorch wheel index alongside the required ML packages in src/backend/requirements.txt:

```
--extra-index-url https://download.pytorch.org/whl/cpu
fastapi
uvicorn
python-multipart
sqlalchemy
python-jose[cryptography]
passlib[bcrypt]
bcrypt==3.2.2
torch
torchvision
anomalib
```

The container was then rebuilt to integrate CPU-bound PyTorch and Anomalib execution natively.

Challenge 4: Cross-Platform Path Conflict (Windows Paths in Linux Docker)

Issue: The backend crashed during inference with "No results folder found for category 'bottle'", referencing a Windows-style drive path.

Root Cause: src/inference/predict.py contained hardcoded Windows drive paths (D:\...) left over from local development, which do not resolve inside a Linux-based Docker container.

Fix Implemented — refactored predict.py to compute paths dynamically and cross-platform, relative to the module's own location, with environment-variable overrides:

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))   # src/inference
SRC_DIR = os.path.abspath(os.path.join(BASE_DIR, ".."))  # src

RESULTS_DIR = os.getenv("RESULTS_DIR", os.path.join(SRC_DIR, "results"))
HEATMAP_OUTPUT_DIR = os.getenv("HEATMAP_OUTPUT_DIR", os.path.join(SRC_DIR, "predictions"))
```

Challenge 5: Frontend Heatmap Rendering & Network Isolation

Issue: Inference completed successfully (returning DEFECT DETECTED or PASS verdicts), but the PatchCore Heatmap Overlay panel sometimes displayed a blank placeholder instead of the actual result image.

Root Cause: Two separate issues contributed to this —

- The backend returned relative image paths, causing the Vite frontend (port 5173) to attempt fetching resources from the wrong origin/port.
- For non-defective (PASS) items, Anomalib does not generate an anomaly map, so heatmap_path returned None.

Fix Implemented:

- Base64 Payload Encoding — updated the /predict route in main.py to read generated heatmap images from disk, encode them as Base64 data URIs, and embed them directly in the JSON payload, removing the cross-origin image request entirely.
- OpenCV Fallback Overlay — added OpenCV logic to generate a visual feed overlay (or a fallback copy of the original image) whenever the model returns a PASS rating without an anomaly map.
- Multi-Key Field Support — standardized the JSON response to expose matching keys (heatmap_url, heatmapUrl, overlay, image_url) to ensure compatibility across frontend state bindings.

5. Files Modified & Updated

File	Primary Modifications
src/backend/requirements.txt	Added CPU-version torch, torchvision, and anomalib dependencies.
src/inference/predict.py	Replaced hardcoded Windows paths with cross-platform relative pathing logic.
src/backend/main.py	Added DB auto-seeding, Base64 image encoding, OpenCV fallback generation, and multi-key JSON response support.

6. Containerized Deployment & Public Access

6.1 Local Deployment

The full platform (frontend, backend, and AI model engine) is started with a single command from the project root:

```
docker compose down -v
docker compose up --build
```

On first startup, the backend automatically seeds two demo accounts for immediate access:

- admin / admin123 — full access, can trigger AI inspections.
- supervisor / supervisor123 — analytics and history access only.

6.2 Public Demo Access

For remote evaluation, the frontend was exposed via a Cloudflare Tunnel:

```
cloudflared tunnel --url http://localhost:5173
```

This generated a temporary public URL — sporting-chains-london-benefit.trycloudflare.com — proxying requests to the local frontend, allowing evaluators to access and test the live system in a browser without any cloud hosting setup.

7. Results & Screenshots

The following screenshots were captured during local testing (post-fix) and after the final Docker-based deployment, demonstrating the resolved issues described in Section 4.

7.1 Local Inspection Testing — Heatmap Rendering

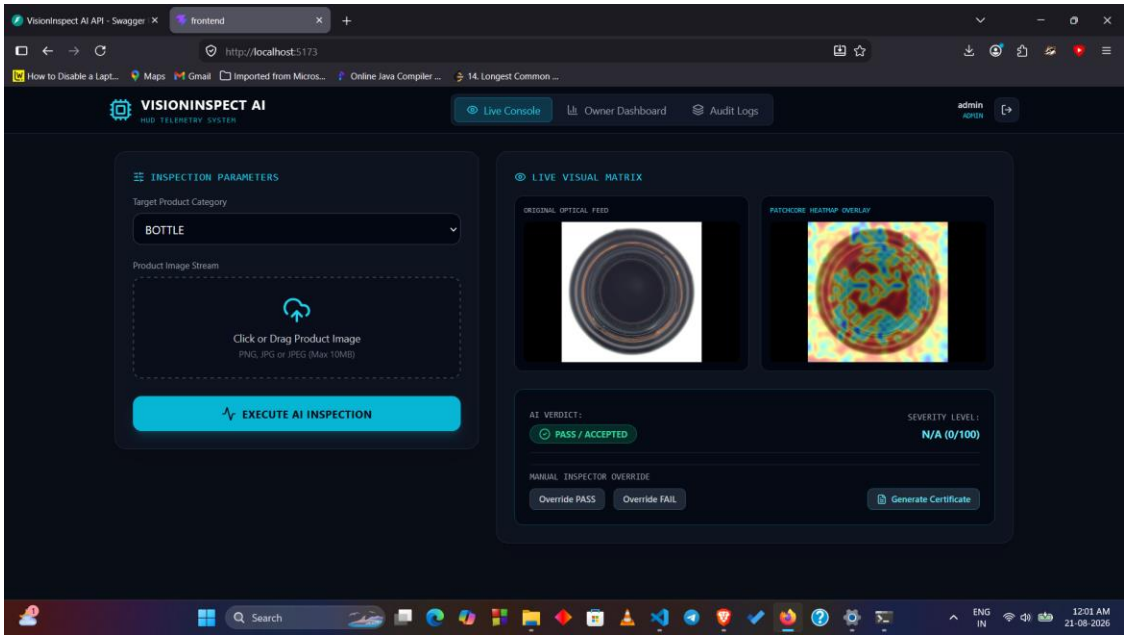

Fig 7.1: Live Inspection Console — Bottle category, PatchCore heatmap overlay rendering correctly alongside the original optical feed.

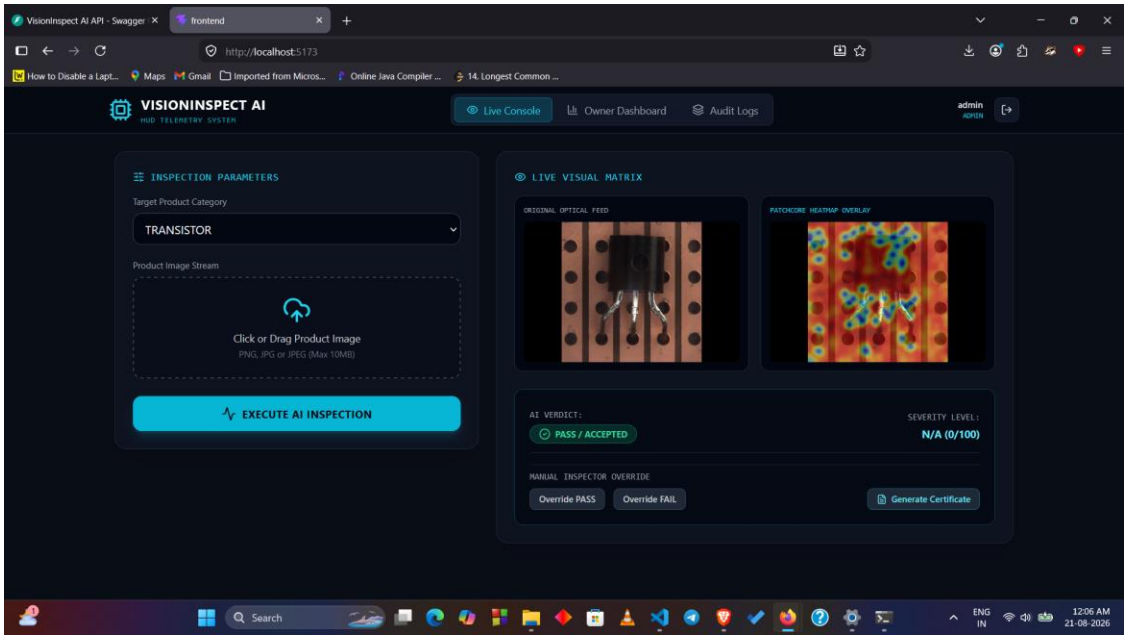

Fig 7.2: Live Inspection Console — Transistor category, heatmap overlay correctly localizing the inspected region.

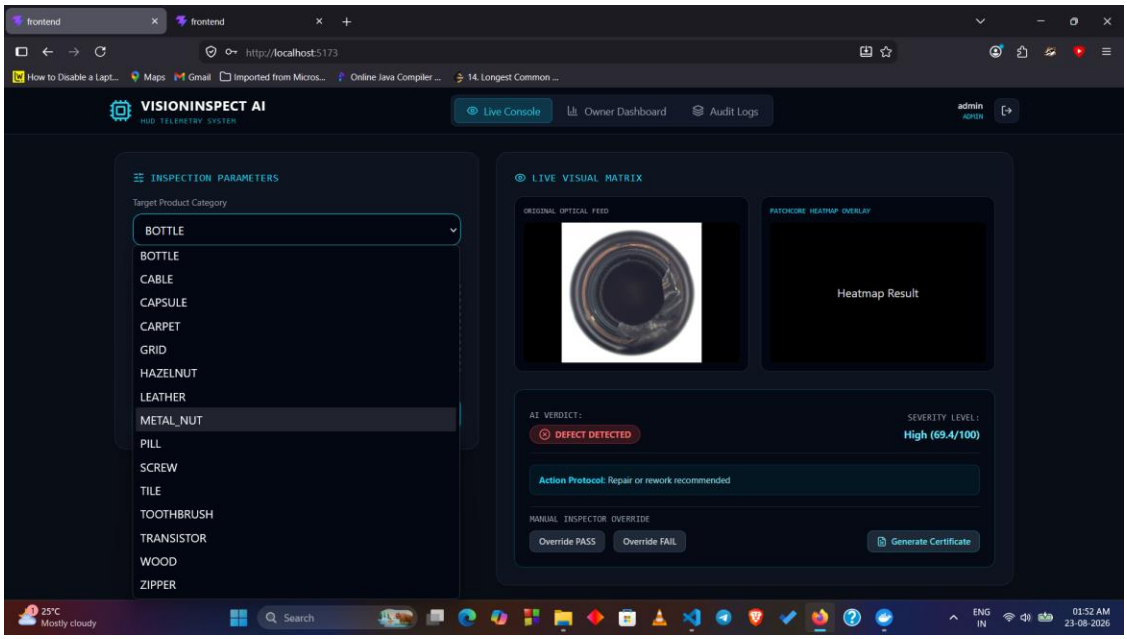

Fig 7.3: Live Inspection Console — category selection dropdown with a DEFECT DETECTED verdict (High, 69.4/100) and the recommended action protocol.

7.2 Owner Dashboard & Analytics

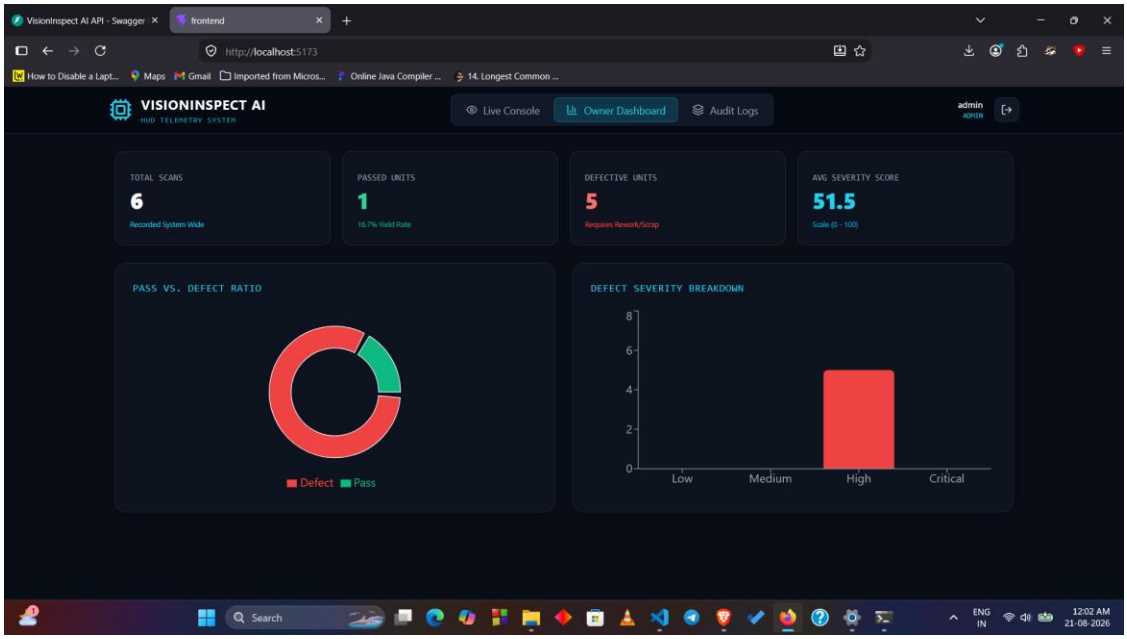

Fig 7.4: Owner Dashboard — inspection statistics after 6 recorded scans, showing Pass/Defect ratio and severity breakdown.

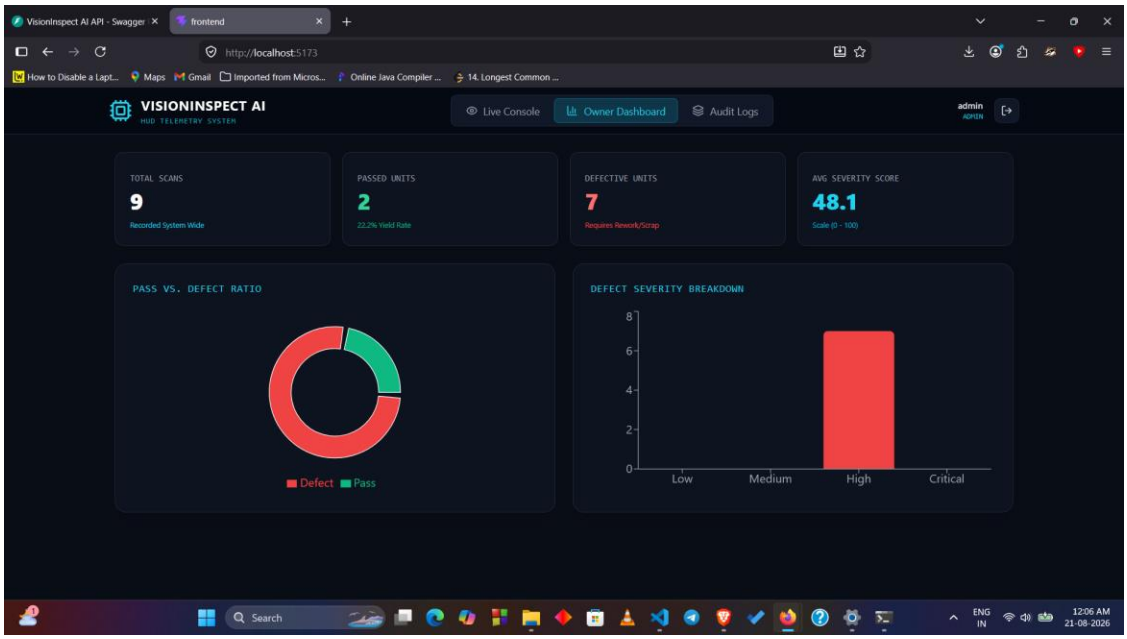

Fig 7.5: Owner Dashboard — updated statistics after further inspections (9 scans), confirming live data binding to the backend.

7.3 Audit Logging

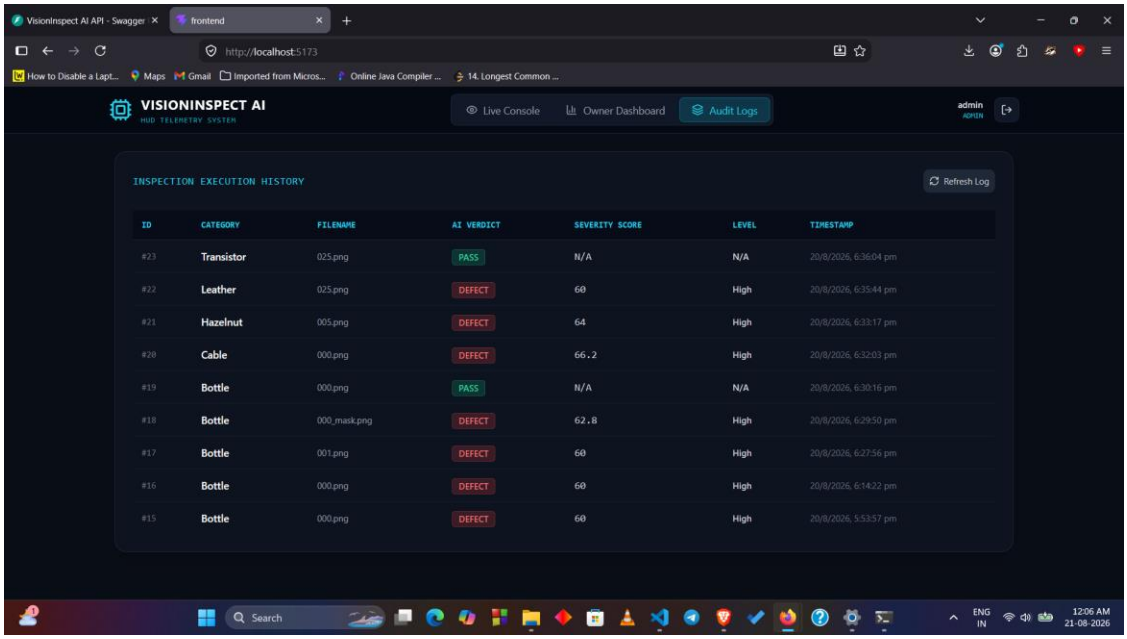

Fig 7.6: Audit Logs — inspection execution history showing category, AI verdict, severity score, and timestamp for each logged inspection.

7.4 Quality Certificate Generation

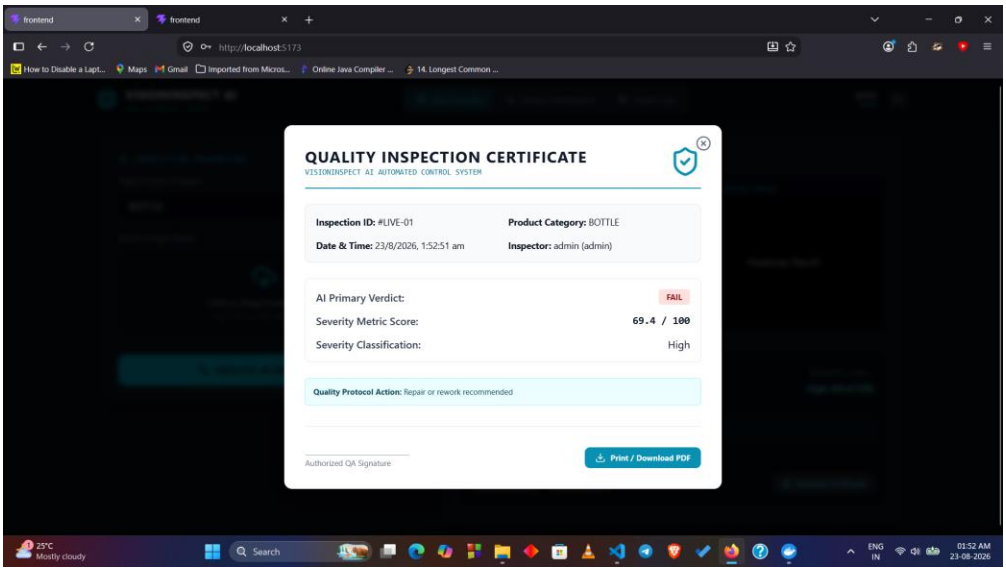

Fig 7.7: Quality Inspection Certificate generated for a FAIL verdict, including inspection ID, severity score, classification, and recommended protocol action.

7.5 Public Deployment via Cloudflare Tunnel

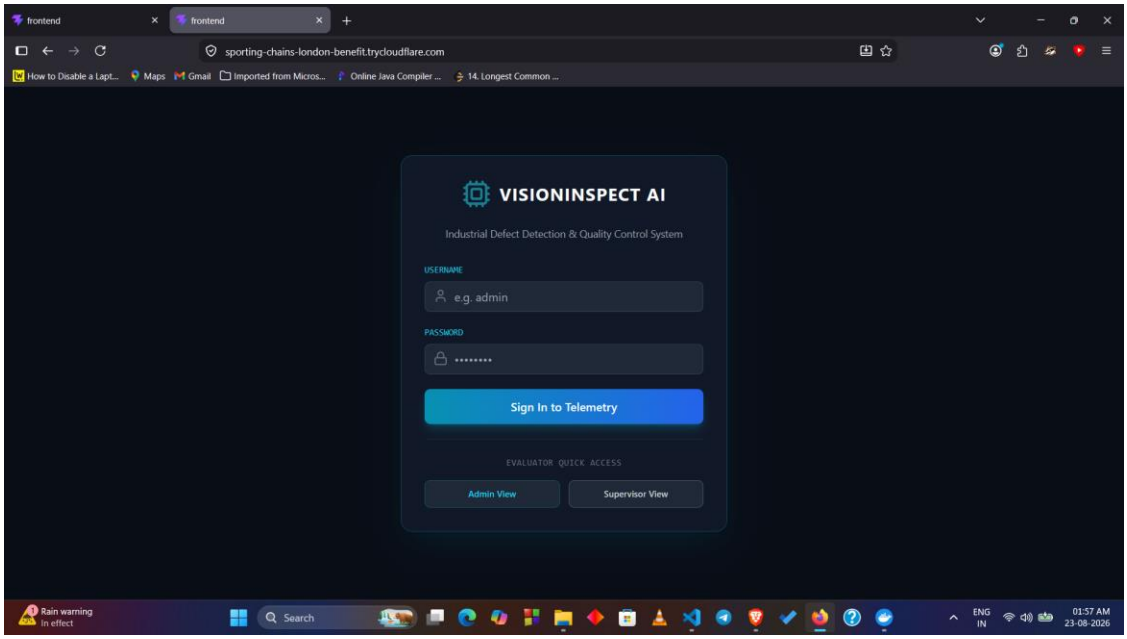

Fig 7.8: VisionInspect AI login screen accessed via the public Cloudflare Tunnel URL (sporting-chains-london-benefit.trycloudflare.com), confirming successful remote deployment.

8. Observations

- Automating demo-account seeding at startup removed a recurring blocker during repeated container rebuilds and evaluator onboarding.
- RBAC scoping needed to be explicitly re-verified after integration, since the default registration role was not sufficient to exercise the AI inspection features end-to-end.
- Keeping heavy ML dependencies (torch, anomalib) out of the base image kept early build iterations fast, but they had to be reintroduced carefully with a CPU-only wheel index to avoid pulling unnecessary CUDA packages inside the container.
- Hardcoded local file-system paths are one of the most common sources of breakage when moving from a Windows development machine to a Linux-based Docker container; resolving paths relative to the module's own location proved to be a robust fix.
- The heatmap rendering issue was really two separate bugs (cross-origin image paths and missing anomaly maps on PASS results) that only became visible together once the frontend and backend were integrated and containerized.
- Exposing the deployed frontend through a Cloudflare Tunnel was an effective way to allow remote evaluation without any cloud infrastructure setup.

9. Conclusion & Future Work

Milestone 4 took the inference and serving layer built in Milestone 3 and turned it into a fully integrated, deployable product. My individual contribution covered backend startup reliability, RBAC verification, dependency resolution for the AI stack, cross-platform path handling, and the frontend heatmap rendering fix — culminating in a fully containerized deployment via Docker Compose and a working public demo via Cloudflare Tunnel.

Recommended Future Work:

- Train a supervised defect-type classifier to replace the current placeholder value used in the severity scoring formula.
- Extend the manufacturing analytics module with defect-trend visualization over time.
- Migrate from SQLite to PostgreSQL to support multi-user production scalability.
- Add an automated CI/CD pipeline for container builds and deployment.

This concludes the Milestone 4 individual contribution report for VisionInspect AI.